

Week 6 lecture notes: Normalisation

Normalisation is how we stop the same fact living in three rows so that an update hits only one of them.

We will use a running example: a table `LoanSheet(memberId, memberName, isbn, title, authorName, copyBarcode, loanedOn)`.

That table is tempting. It is also a trap.

1NF

Atomic values, no repeating groups. `authorName` as “Ng, Knuth” is already a 1NF problem if you ever want to query by one author. Split authors out. Repeating `isbn1`, `isbn2`, `isbn3` columns is the other classic failure.

2NF

Non-key attributes must depend on the *whole* key. If the key is (`memberId`, `isbn`) and `memberName` depends only on `memberId`, you are in 1NF but not 2NF. Pull member data into `Member`.

3NF

Non-key attributes must not depend on other non-key attributes. `title` depends on `isbn`, not on the loan. `Loan` should hold `copyBarcode` and `loanedOn`. `Copy` points at `Book`. `Book` holds `title`.

After 3NF you should be able to change a title in one place and have every loan printout follow.

BCNF is the extra step when a table has overlapping candidate keys. We will not examine BCNF in depth. If you can get to 3NF without losing a dependency, you are where this course needs you.

Mini-project

Take the denormalised `events.csv` on the course page (festival tickets, one row per sale). Produce 3NF relations, a short note of which anomalies disappeared, and the SQL `CREATE TABLE` statements.

The normalisation mini-project is due 20 November 2026. Submit SQL plus a one-page PDF. This is individual work.

If you only rename the wide table and call it 3NF, we will send it back. Show the functional dependencies.